

Concepts at a Glance

MC vs TD vs n -step			
	MC	TD(0)	n -step
Target	G_t	$R + \gamma V(S')$	mixed
Bias	None	Yes	Med
Var	High	Low	Med
Episodes?	Must end	Online	Online
Bootstrap?	No	Yes	Yes

On-Policy vs Off-Policy

Behavior b = policy collecting data

Target π = policy being improved

On-policy: $b = \pi$ (SARSA, MC control)

Off-policy: $b \neq \pi$ (Q-learning)

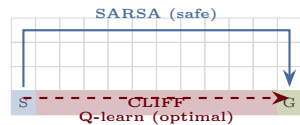
ε -Greedy

Greedy action: $1 - \varepsilon + \varepsilon/|\mathcal{A}|$

Other actions: $\varepsilon/|\mathcal{A}|$ each

Why? Pure greedy gets stuck; ε -greedy explores.

SARSA vs Q-Learning



SARSA: cautious (accounts for ε -exploration)

Q-learn: optimal (assumes greedy from now on)

Bootstrapping

Using *current estimates* to update estimates.

TD: yes $\rightarrow$ biased but fast. MC: no $\rightarrow$ unbiased but slow.

n -Step & TD(λ)

$n = 1$: TD(0). $n = \infty$: MC. Sweet spot ≈ 5 .

TD(λ): weighted avg of all n -step returns. $\lambda = 0 \rightarrow$ TD(0). $\lambda = 1 \rightarrow$ MC.

Why Q not V ?

Can't pick actions from $V(s)$ without model $p(s'|s, a)$.

$Q(s, a)$ lets you do $\pi(s) = \arg \max_a Q(s, a)$ directly.

Deep Learning

Neural Networks
$f(\mathbf{x}) = \mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}$ Non-linear activations required! Without them: $\mathbf{W}_2 \mathbf{W}_1 \mathbf{x} = \mathbf{W}' \mathbf{x}$ (still linear)

Activation Functions

Sigmoid: $\frac{1}{1+e^{-x}} \rightarrow$ vanishing grad

Tanh: output $(-1, 1)$ ReLU: $\max(0, x)$ most common

Leaky ReLU: $\max(0.01x, x)$ fixes dead neurons

Overfitting Fixes

Regularization ($\lambda \sum W^2$), dropout, data augmentation, early stopping, simpler model

Optimizers

SGD: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla E$

Momentum: $\mathbf{v} \leftarrow \mu \mathbf{v} + \alpha \nabla E$; $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{v}$

ADAM: adaptive lr per param (mean + variance)

ResNets & BatchNorm

Skip connection: output = $F(\mathbf{x}) + \mathbf{x}$

Gradient: $\frac{\partial}{\partial \mathbf{x}} [F(\mathbf{x}) + \mathbf{x}] = F' + \mathbf{I} (\geq 1, \text{ no vanishing})$

BatchNorm: normalize to $\mu = 0, \sigma = 1$ per batch

Hyperparams vs Params

Params: learned (weights, biases)

Hyperparams: set before training (lr, layers, ε)

PyTorch

Training Loop (memorize order!)
<code>optimizer.zero_grad()</code> clear ∇ <code>out = nnet(x)</code> forward <code>loss = criterion(out, y)</code> <code>loss.backward()</code> compute $\nabla_{\theta} \mathcal{L}$ <code>optimizer.step()</code> $\theta \leftarrow \theta - \alpha \nabla$

`.detach()` = stop gradient flow (for targets)

`loss.item()` = get scalar (prevent memory leak)

`.train()` = dropout ON. `.eval()` =

dropout OFF

`torch.save(nnet.state_dict(), ...)`

= save params only

DQN

Three DQN Innovations
Target network θ^-: freeze target for C steps Experience replay: store (s, a, r, s'), sample random batches (breaks correlation) Huber loss: clips gradients for large TD errors Ablation: removing <i>either</i> replay OR target net $\rightarrow$ catastrophic failure

DQN Improvements

Double Q: fix max bias—one net picks, other evaluates

Dueling: $q = v(s) + A(s, a)$; separates state value from action advantage

Prioritized replay: sample large-error transitions more

Rainbow: all combined. **Agent57:** beats all 57 Atari

Math: All Derivations

1. MC Incremental Update

Start: $V_n = \frac{1}{n} \sum_{k=1}^n G_k$. Split off n -th term:

$$V_n = \frac{1}{n} \left(G_n + (n-1)V_{n-1} \right)$$
$$= V_{n-1} + \frac{1}{n} (G_n - V_{n-1})$$

Trick: rewrite $\frac{n-1}{n} = 1 - \frac{1}{n}$, then collect terms.

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$

2. ϵ -Greedy Policy Improvement

Show $q_\pi(s, \pi'(s)) \geq v_\pi(s)$:

$$\sum_a \pi'(a|s) q_\pi(s, a)$$
$$= \frac{\epsilon}{|\mathcal{A}|} \sum_a q_\pi(s, a) + (1-\epsilon) \max_a q_\pi(s, a)$$

Since $\max \geq$ weighted avg, substitute weights $w_a = \frac{\pi(a|s) - \epsilon/|\mathcal{A}|}{1-\epsilon}$:

$$\geq \frac{\epsilon}{|\mathcal{A}|} \sum_a q_\pi + \sum_a (\pi(a|s) - \epsilon/|\mathcal{A}|) q_\pi$$
$$= \sum_a \pi(a|s) q_\pi(s, a) = v_\pi(s) \quad \square$$

3. Sigmoid Derivative

$$\sigma(z) = (1 + e^{-z})^{-1}$$
$$\sigma' = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}$$
$$= \sigma \cdot \frac{(1 + e^{-z}) - 1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Max at $z=0$: $\sigma' = 0.25 \rightarrow$ causes vanishing gradients.

4. Logistic Regression Gradient

$$\ell = y \log \hat{y} + (1-y) \log(1-\hat{y}), \quad \hat{y} = \sigma(\mathbf{w}^\top \mathbf{x})$$
$$\frac{\partial \ell}{\partial \mathbf{w}} = y \frac{\hat{y}(1-\hat{y})}{\hat{y}} \mathbf{x} + (1-y) \frac{-\hat{y}(1-\hat{y})}{1-\hat{y}} \mathbf{x}$$
$$= (y - \hat{y}) \mathbf{x}$$

Key: $\sigma' = \sigma(1-\sigma)$ cancels denominators!

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N} \sum_i (\hat{y}_i - y_i) \mathbf{x}_i$$

5. Backprop (2-Layer Network)

$$\hat{\mathbf{y}} = \mathbf{W}^{(2)} \mathbf{h}, \quad \mathbf{h} = \sigma(\mathbf{W}^{(1)} \mathbf{x}), \quad E = \frac{1}{2} \|\mathbf{y} - \hat{\mathbf{y}}\|^2$$

Output weights:

$$\frac{\partial E}{\partial W_{ij}^{(2)}} = -(y_i - \hat{y}_i) \cdot h_j$$

Hidden weights: (chain through 2 layers)

$$\frac{\partial E}{\partial W_{jk}^{(1)}} = \underbrace{\sum_i [-(y_i - \hat{y}_i)] W_{ij}^{(2)}}_{\text{backprop error}} \sigma'(\cdot) x_k$$

Pattern: error $\times$ connecting weight $\times \sigma' \times$ input

6. DAVI Gradient

$$y = \max_a [r(s, a) + \gamma \sum_{s'} p(s'|s, a) v_{\theta-}(s')] \text{ (constant!)}$$
$$L(\theta) = \frac{1}{2} (y - v_\theta(s))^2$$
$$\nabla_\theta L = -(y - v_\theta(s)) \nabla_\theta v_\theta(s)$$

$$\theta \leftarrow \theta + \alpha (y - v_\theta(s)) \nabla_\theta v_\theta(s)$$

Do NOT differentiate y w.r.t. θ ! y uses θ^- (frozen).
PyTorch: compute y with `.detach()` or `no_grad()`.

7. Linear Regression Closed Form

$$\mathcal{L} = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{w}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}$$

Set $\nabla_{\mathbf{w}} \mathcal{L} = 2\mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{X}^\top \mathbf{y} = \mathbf{0}$

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Key identity: $\nabla_{\mathbf{w}} (\mathbf{w}^\top \mathbf{A} \mathbf{w}) = 2\mathbf{A} \mathbf{w}$

! Common Derivation Tricks:

- Split off last term from sum: $\sum_1^n = \sum_1^{n-1} + n\text{-th}$
- $\sigma' = \sigma(1-\sigma)$ cancels log denominators
- $\max_a f(a) \geq \sum_a w_a f(a)$ for any convex weights
- Chain rule = multiply derivatives layer by layer
- Matrix: $\nabla_{\mathbf{w}} (\mathbf{w}^\top \mathbf{A} \mathbf{w}) = 2\mathbf{A} \mathbf{w}$
- Never differentiate the target y in DAVI/DQN

Key Update Equations

MC: $V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$

TD(0): $V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R + \gamma V(S'_t) - V(S_t)]}_{\text{target}}$

SARSA: $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

Q-learn: $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

DQN: $y = R + \gamma \max_{a'} q_{\theta-}(S', a')$
 $L = \frac{1}{2} (y - q_\theta(S, A))^2$

n-step: $G_{t:t+n} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n})$

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \quad (\text{TD error})$$
$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s) \quad (\text{advantage})$$

Robbins-Monro: $\sum \alpha_n = \infty, \sum \alpha_n^2 < \infty$